

A MINI PROJECT REPORT ON

STUDENT RANKING SYSTEM

Using Heap and Sorting Algorithms

Submitted in partial fulfilment of the requirements for the Mini Project

Submitted by

T.Likhitha

Roll No: 23AK1A3341

[AIML/ 3 / 1]

[AIML]

[AITS TPT]

[2023-2027]

Table of Contents

1. Abstract	3
2. Introduction	3
3. Objectives.....	4
4. Technologies Used	4
5. Methodology.....	5
5.1 System Architecture	5
5.2 Heap Design	5
5.3 Sorting Algorithms.....	5
5.4 Program Workflow.....	6
6. Results	6
7. Conclusion	7

1. Abstract

This project presents the design and implementation of a Student Ranking System that combines heap-based data structures with classical sorting algorithms to efficiently rank students based on academic performance. Conventional ranking systems typically rely on a single sorting technique, which can become computationally expensive as the dataset grows or when only the top-performing students need to be identified.

To address this limitation, the proposed system implements four widely used sorting algorithms — Bubble Sort, Merge Sort, Quick Sort, and Heap Sort — enabling a comparative study of their behaviour and time complexity, alongside a custom-built Max-Heap data structure that retrieves the top K students without sorting the entire dataset.

The system is implemented in Java using object-oriented programming principles and provides a menu-driven console interface for adding student records, viewing the complete list, generating rankings, and extracting top performers. Execution time for each sorting technique is measured and displayed at runtime, helping illustrate algorithmic efficiency in a practical, hands-on context. The project therefore serves both as a functional ranking utility and as an educational demonstration of the trade-offs between full-list sorting and heap-based selection.

2. Introduction

Ranking students based on their academic performance is a routine yet important task in educational institutions. It is used for generating merit lists, awarding scholarships, shortlisting candidates for further opportunities, and providing students with feedback on their relative standing. As the number of students in a class, department, or institution grows, manually computing and maintaining such rankings becomes impractical, motivating the need for an automated, efficient, and scalable solution.

From a computer science perspective, ranking is fundamentally a sorting problem. While many programming languages provide built-in sorting utilities, understanding how different sorting algorithms work internally — and how their time complexities differ — is an essential part of learning data structures and algorithms (DSA). Additionally, in many practical scenarios, an institution may only be interested in a small subset of top performers (for example, the top 10 students for an award) rather than a fully sorted list of the entire class. Computing a full sort purely to extract a few top entries is often wasteful; a heap-based selection approach is more appropriate for such cases.

This project, the Student Ranking System, was developed to explore both aspects together. It allows a user to add student records (roll number, name, and marks), and then choose between four classical sorting algorithms — Bubble Sort, Merge Sort, Quick Sort, and Heap Sort — to generate a full ranked list, with the time taken by each algorithm displayed for comparison.

Separately, the system implements a custom array-based Max-Heap that can efficiently retrieve only the top K students, demonstrating an alternative, more efficient approach when a full ranking is not required.

The remainder of this report is organised as follows: Section 3 outlines the objectives of the project, Section 4 lists the technologies used, Section 5 describes the methodology and system design in detail, Section 6 presents sample results and discussion, and Section 7 concludes the report with a summary and directions for future enhancement.

3. Objectives

The key objectives of this mini project are as follows:

- To design and implement a Java-based console application for managing and ranking student records.
- To implement and compare four classical sorting algorithms — Bubble Sort, Merge Sort, Quick Sort, and Heap Sort — for ranking students by marks.
- To design a custom Max-Heap data structure from first principles (without relying on Java's built-in PriorityQueue) for efficient top-K element retrieval.
- To analyse and demonstrate the practical difference in performance between full-list sorting and heap-based partial selection.
- To apply object-oriented programming concepts such as encapsulation and modular class design to build a maintainable, extensible application.
- To provide a simple, interactive, menu-driven user interface suitable for academic demonstration.

4. Technologies Used

The project was built entirely using core Java, without external frameworks, so that the focus remains on data structures and algorithm design rather than tooling. The table below summarises the technologies and concepts used.

Category	Details
Programming Language	Java (JDK 8 or above)
Core Concepts	Object-Oriented Programming (Classes, Encapsulation)
Data Structures	Arrays, ArrayList, Custom Max-Heap
Algorithms	Bubble Sort, Merge Sort, Quick Sort, Heap Sort
Java Libraries	java.util (Scanner, ArrayList, Arrays, List)

Development Tools	Any text editor / IDE (VS Code, Eclipse, IntelliJ IDEA) with JDK and javac/java CLI
Interface	Console-based (Command Line Interface)

5. Methodology

5.1 System Architecture

The system follows a simple modular design consisting of four classes, each with a single, well-defined responsibility:

- Student — a data class storing roll number, name, and marks for one student.
- MaxHeap — a custom array-based max-heap supporting insert(), extractMax(), and getTopK(k), used for efficient top-performer retrieval.
- Sorter — a utility class containing static implementations of Bubble Sort, Merge Sort, Quick Sort, and Heap Sort, each adapted to sort Student objects in descending order of marks.
- StudentRankingSystem — the main class providing a menu-driven console interface that ties the other classes together.

5.2 Heap Design

The Max-Heap is implemented using a dynamically resizable array, where the parent of index i is at $(i-1)/2$, and its children are at $2i+1$ and $2i+2$. Insertion places a new element at the end of the array and repeatedly swaps it with its parent (“bubble up”) until the heap property — every parent's marks are greater than or equal to its children's marks — is restored. Extraction removes the root (maximum), replaces it with the last element, and repeatedly swaps it down (“heapify”) until the heap property is restored again. Both operations run in $O(\log n)$ time, which makes repeated extraction of the top K students significantly cheaper than sorting the entire list when K is small.

5.3 Sorting Algorithms

Each sorting algorithm operates on an array of Student objects and compares students by their marks field, arranging them in descending order so that Rank 1 corresponds to the highest scorer:

- Bubble Sort repeatedly steps through the list, swapping adjacent elements that are in the wrong order, until no swaps are needed.
- Merge Sort recursively divides the list into halves, sorts each half, and merges the two sorted halves back together.
- Quick Sort selects a pivot element and partitions the array so that all elements greater than or equal to the pivot come before it, then recursively sorts the partitions.

- Heap Sort builds a max-heap from the array and repeatedly extracts the maximum element to build the sorted sequence.

The theoretical time and space complexities of all algorithms used in this project are summarised below:

Algorithm	Best Case	Average Case	Worst Case	Space
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Max-Heap Insert / Extract	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$

5.4 Program Workflow

The application presents the user with a menu offering five options: add a student record, display all records, rank students using a chosen sorting algorithm (with execution time displayed), retrieve the top K students using the Max-Heap, and exit. The program loops on this menu until the user chooses to exit, allowing repeated insertion and ranking operations within a single session.

6. Results

The application was tested by adding multiple student records and exercising each menu option. A representative sample run, using Quick Sort to rank three students, is shown below:

```
===== Student Ranking System =====
1. Add Student
2. Display All Students
3. Rank Students (Choose Sorting Algorithm)
4. Get Top K Students (Using Max-Heap)
5. Exit
Enter choice: 3
Choose Sorting Algorithm:
1. Bubble Sort
2. Merge Sort
3. Quick Sort
4. Heap Sort
Enter choice: 3

--- Ranking (Highest to Lowest Marks) ---
Rank 1: Roll No: 102 | Name: Kiran | Marks: 95.0
Rank 2: Roll No: 101 | Name: Ravi | Marks: 88.0
Rank 3: Roll No: 103 | Name: Sai | Marks: 70.0
```

```
Time taken: 4300 ns
```

All four sorting algorithms produced identical, correctly ordered rankings for every test dataset, confirming functional correctness. As expected from their theoretical complexities, Bubble Sort consistently took noticeably longer than Merge Sort, Quick Sort, and Heap Sort as the number of records increased, while the three $O(n \log n)$ algorithms performed comparably for the dataset sizes tested. The “Top K using Max-Heap” option correctly returned the K highest-scoring students without sorting the full list, and its underlying insert/extract operations completed in $O(\log n)$ time per element as expected, making it noticeably more efficient than a full sort when K was small relative to the total number of students.

These results validate that the implementation behaves consistently with the theoretical time-complexity expectations discussed in Section 5, and demonstrate the practical advantage of heap-based selection for top-K queries over full-list sorting.

7. Conclusion

This mini project successfully demonstrates the design and implementation of a Student Ranking System in Java that integrates a custom Max-Heap with four classical sorting algorithms — Bubble Sort, Merge Sort, Quick Sort, and Heap Sort. The system allows students to be ranked by marks using any of the four algorithms, while also supporting efficient retrieval of the top K performers via the heap, without the overhead of sorting the entire dataset.

Through this project, practical experience was gained in object-oriented design, array-based data structure implementation, recursive and iterative algorithm design, and empirical performance comparison of algorithms with differing theoretical complexities. The project reinforces core data structures and algorithms concepts in a tangible, real-world-style application rather than as isolated theoretical exercises.

As future enhancements, the system could be extended with a graphical user interface (e.g., using Java Swing or JavaFX), persistent storage using file handling or a database such as MySQL via JDBC, support for ranking by multiple subjects with weighted averages, and additional sorting algorithms such as Tim Sort or Radix Sort for further comparative analysis.